TurboRAG

Compressed Vector-Graph Retrieval Library

Technical Architecture & Agent Build Specification

Ratnam | March 2026

1. Executive Summary

TurboRAG is a Python library that combines two independently powerful advances — TurboQuant's near-lossless vector compression and Microsoft GraphRAG's knowledge-graph-augmented retrieval — into a single, drop-in package. Neither project has attempted this combination. The result is a retrieval system that is simultaneously more memory-efficient than any current vector store, more semantically accurate than flat similarity search, and capable of multi-hop reasoning that pure embedding approaches cannot do at all.

This document is a complete technical specification for a coding agent to implement TurboRAG from scratch. It covers the mathematical foundations, module architecture, every data structure and algorithm, distribution strategy, and a detailed comparison against PageIndex — the closest conceptual neighbor.

2. The Problem Space

2.1 Why existing RAG is broken at scale

The retrieval step in RAG is fundamentally a maximum inner product search (MIPS) problem: given query vector q and a corpus of N stored vectors {v_1 ... v_N}, find the k vectors that maximise dot(q, v_i). Three compounding forces make this expensive:

- Storage explosion: N vectors × D dimensions × 4 bytes (float32). At N=10M, D=1536 (OpenAI ada-002), this is 61 GB. At D=3072 (text-embedding-3-large) it is 123 GB — beyond a single A100.
- Graph overhead: HNSW — the gold-standard ANN index — stores 32–64 bidirectional pointers per vector plus layer metadata, adding 2–4× memory on top of raw vectors. At 10M vectors with D=1536 this is 180–370 GB total.
- Compression accuracy loss: IVF-PQ, the standard compression method, breaks vectors into M sub-vectors and quantizes each to a codebook of K centroids. This introduces systematic bias — certain directions in embedding space are compressed more aggressively, distorting dot product similarity. Recall@10 degradation of 5–15% is typical, which translates to wrong context chunks fed to the LLM in ways that are silent and hard to debug.

2.2 Why GraphRAG alone is not enough

Microsoft's GraphRAG extracts entities and relationships from documents, builds a knowledge graph, computes community summaries via the Leiden algorithm, and routes queries through that graph for multi-hop reasoning. It achieves genuinely superior performance on queries that require connecting facts across documents (e.g. 'What did the CFO say about capital allocation in both the Q2 and Q3 earnings calls?').

However GraphRAG still embeds every entity, relationship, and community summary into a standard float32 vector store for its final similarity lookup step. It inherits the full storage explosion problem. Additionally it requires many LLM calls during index construction (entity extraction, summarisation), which is expensive and slow. It does not compress anything — it adds a reasoning layer on top of the same broken storage layer.

Key Insight	TurboQuant and GraphRAG are orthogonal. TurboQuant fixes the storage layer. GraphRAG fixes the reasoning layer. No existing system does both. TurboRAG does.

3. Mathematical Foundations

3.1 PolarQuant — the rotation step

Given an input vector $x \in R^D$, PolarQuant applies a random orthogonal rotation matrix $R \in R^{D\times D}$ drawn from the Haar distribution (uniformly random rotation), producing $x' = Rx$. The key property: for a random Gaussian vector x, the coordinates of Rx concentrate tightly around $\pm 1/\sqrt{D}$ after normalisation. This means the rotated coordinates are near-uniform in magnitude, making them ideal for uniform quantization — there are no large outlier values that would waste quantization range.

The rotation matrix R is generated once per index using a random seed, stored as a compact representation (a seed + algorithm ID, not the full D×D matrix, which would be larger than the vectors themselves). At query time the same R is applied to the query vector before similarity computation.

3.2 QJL — the quantization step

After rotation, QJL (Quantized Johnson-Lindenstrauss) applies a structured random projection $S \in R^{m\times D}$ where $m < D$, followed by a sign operation. Specifically, for each projected dimension j, the quantized bit $b_j = \text{sign}(S_j \cdot x')$. The projection S is constructed as a product of a random ±1 diagonal matrix and a Hadamard matrix (SRHT — subsampled randomized Hadamard transform), enabling $O(D \log D)$ application instead of $O(mD)$.

The Johnson-Lindenstrauss lemma guarantees that for any two vectors u, v and projection dimension $m \geq O(\log(1/\delta) / \varepsilon^2)$, with probability $\geq 1-\delta$: $|\text{dot}(u,v) - \text{dot}(u_compressed, v_compressed)| \leq \varepsilon \cdot ||u|| \cdot ||v||$. This is a provable error bound, not an empirical approximation. PQ has no equivalent guarantee — its error depends on codebook quality and can be arbitrarily bad for out-of-distribution inputs.

3.3 Bit packing and storage layout

At 3 bits per dimension for D=1536, each vector occupies $\lceil 1536 \times 3 / 8 \rceil = 576$ bytes, versus 6144 bytes for float32 — a 10.67× reduction in raw size. In practice TurboQuant achieves ~6× after accounting for the stored rotation seed and metadata per shard. Dot products in compressed space are computed as popcount operations on bitwise-packed integers, which are dramatically faster than float multiply-accumulate chains and benefit from SIMD acceleration on all modern CPUs and GPUs.

4. System Architecture

4.1 Module overview

Module	Responsibility
turborag.compress	PolarQuant rotation and QJL quantization. Stateless functions that operate on numpy arrays. No ML dependencies.
turborag.index	TurboIndex class — the compressed vector store. Wraps sharded on-disk storage with MMAP access, supports add(), search(), and save()/load().
turborag.graph	GraphBuilder class — entity extraction, relationship detection, community detection (Leiden), and summary generation. Wraps an LLM client.
turborag.hybrid	HybridRetriever — the main user-facing class. Combines TurboIndex for dense retrieval with the graph layer for multi-hop reasoning. Single query() method.
turborag.ingest	Document ingestion pipeline: chunking, embedding, compression, graph extraction. Handles PDF, markdown, plain text.
turborag.bench	Benchmarking utilities. Measures recall@k, query latency, memory footprint. Outputs comparison tables vs HNSW and IVF-PQ.
turborag.serve	Optional FastAPI wrapper exposing /ingest, /query, and /graph endpoints. Used when deploying as a service rather than a library.

4.2 Data flow — index time

The full indexing pipeline from raw document to queryable compressed index follows these steps in sequence:

1. Document ingestion: raw text/PDF/markdown is loaded and split into chunks. Default chunk size is 512 tokens with 64-token overlap. Each chunk is assigned a UUID and stored in a SQLite metadata table alongside its source document path, page number, and section heading (if available).
2. Embedding: chunks are passed to the configured embedding model (default: sentence-transformers/all-MiniLM-L6-v2 for local, OpenAI text-embedding-3-small for API). Output is a numpy array of shape (N, D) in float32.
3. PolarQuant rotation: the rotation matrix R is loaded from the index config (or generated and saved if this is the first batch). The numpy operation is X_rotated = X @ R.T, executed in fp32 for numerical stability.
4. QJL quantization: X_rotated is projected through the SRHT matrix S and sign-quantized. Output is a bit-packed array of shape (N, ⌈D_projected × bits / 8⌉) as uint8. This is stored in a memory-mapped .bin shard file.
5. Graph extraction: in parallel (or sequentially if no GPU), an LLM extracts entity-relationship triples from each chunk. These are stored in a NetworkX graph object, serialised to GraphML. Communities are detected using the Leiden algorithm (leidenalg library).
6. Community summarisation: each community (cluster of semantically related entities) is summarised by the LLM into a short paragraph. These summaries are themselves embedded and compressed, stored in a separate summary shard.

4.3 Data flow — query time

7. Query embedding: the query string is embedded using the same model as indexing, producing q ∈ R^D.
8. Query compression: R is applied (q_rotated = q @ R.T), then QJL produces q_compressed.
9. Dense retrieval: popcount-based dot product between q_compressed and all stored compressed vectors, returning top-K chunk UUIDs by score.
10. Graph routing: if the query contains named entities that appear in the graph, a secondary search traverses the entity graph using BFS up to depth 2, collecting related entities and their community summaries.
11. Re-ranking: the union of dense results and graph-routed results is re-ranked by a cross-encoder (default: cross-encoder/ms-marco-MiniLM-L-6-v2). Top-k_final results are returned.
12. Context assembly: retrieved chunks and relevant community summaries are concatenated and passed to the LLM with the query.

5. Key Classes and Interfaces

5.1 TurboIndex

The core vector store. Designed to be a drop-in replacement for FAISS's IndexFlatIP with 6× lower memory and faster query throughput via bitwise dot product.

```
class TurboIndex:

    def __init__(self, dim: int, bits: int = 3, shard_size: int = 100_000,

                 storage_dir: str = None, seed: int = 42):

        # dim: embedding dimension

        # bits: quantization bits per dimension (2, 3, or 4)

        # shard_size: vectors per shard file (controls memory vs file count)

        # storage_dir: path for mmap'd shard files; None = in-memory

        # seed: seed for rotation matrix generation



    def add(self, vectors: np.ndarray, ids: List[str]) -> None:

        # vectors: float32 array shape (N, dim)

        # ids: list of N string UUIDs



    def search(self, query: np.ndarray, k: int = 10) -> List[Tuple[str,
float]]:

        # query: float32 array shape (dim,)

        # returns: list of (id, score) sorted by descending score



    def save(self, path: str) -> None: ...

    def load(self, path: str) -> None: ...

    def __len__(self) -> int: ...  # number of indexed vectors
```

5.2 GraphBuilder

Builds the knowledge graph from document chunks. Entity extraction uses a structured LLM prompt that returns JSON. Relationship detection is a second LLM call. Both calls are cached by chunk UUID to avoid re-processing on re-index.

```
class GraphBuilder:

    def __init__(self, llm_client, entity_types: List[str] = None,

                 max_community_size: int = 10, cache_dir: str = None):

        # llm_client: any client with .complete(prompt) -> str method

        # entity_types: if None, auto-detect; else filter to these types

        # cache_dir: path for LLM call cache (SQLite)



    def add_chunk(self, chunk_id: str, text: str) -> None:

        # extracts entities and relationships, adds to internal graph



    def build(self) -> nx.Graph:

        # runs Leiden community detection, assigns community IDs

        # returns the full NetworkX graph



    def get_communities(self) -> Dict[int, List[str]]:

        # returns {community_id: [entity_name, ...]}



    def summarise_communities(self) -> Dict[int, str]:

        # calls LLM to summarise each community

        # returns {community_id: summary_text}
```

5.3 HybridRetriever — the main user-facing class

```
class HybridRetriever:

    def __init__(self, index: TurboIndex, graph: nx.Graph,

                 community_summaries: Dict[int, str],

                 embedder, reranker=None, graph_depth: int = 2):



    def query(self, text: str, k: int = 10,

              mode: str = 'hybrid') -> List[RetrievalResult]:

        # mode: 'dense' | 'graph' | 'hybrid'

        # RetrievalResult: dataclass with chunk_id, text, score,

        #   source_doc, page_num, graph_path (if graph-routed)



    def explain(self, result: RetrievalResult) -> str:

        # returns human-readable explanation of why this result was retrieved

        # e.g. 'Matched entity CFO -> capital_allocation -> Q3_earnings'
```

6. On-Disk Storage Format

A TurboRAG index is a directory with the following layout:

```
my_index/

  config.json           # dim, bits, seed, embedding model ID, schema version

  rotation.npy          # stored rotation matrix R (float32, D×D)
```

```
  shards/
    shard_000.bin       # bit-packed compressed vectors, mmap-accessible
    shard_000.ids.json  # ordered list of UUIDs for this shard
    shard_001.bin
    shard_001.ids.json
  metadata.db           # SQLite: chunk_id, source_doc, page, section,
raw_text
  graph.graphml         # NetworkX-serialised entity-relationship graph
  communities.json      # {community_id: {entities: [...], summary: '...'}}
  summary_shard.bin     # compressed vectors of community summaries
```

The shard .bin files are raw byte arrays accessed via numpy memmap — no serialisation overhead on load. Each vector occupies exactly ⌈D × bits / 8⌉ bytes. Shards are kept at ≤100K vectors by default, bounding peak RAM during search to a controllable size regardless of total index scale.

7. Compression Implementation Detail

7.1 Rotation matrix generation

```
import numpy as np
from scipy.stats import ortho_group


def generate_rotation(dim: int, seed: int) -> np.ndarray:
    rng = np.random.default_rng(seed)
    # For large D, use a product of two Householder reflections
    # (cheaper than full Haar distribution for D > 2048)
```

```
    if dim <= 2048:
        return ortho_group.rvs(dim, random_state=int(seed))
    else:
        # Randomised block-diagonal rotation
        block_size = 512
        blocks = [ortho_group.rvs(block_size, random_state=int(seed)+i)
                  for i in range(dim // block_size)]
        return scipy.linalg.block_diag(*blocks)
```

7.2 QJL quantization (3-bit example)

```
def quantize_qjl(x_rotated: np.ndarray, bits: int = 3) -> np.ndarray:
    # x_rotated: (N, D) float32
    N, D = x_rotated.shape
    levels = 2**bits  # 8 levels for 3-bit
    # Compute per-dimension min/max across batch for calibration
    lo = x_rotated.min(axis=0, keepdims=True)
    hi = x_rotated.max(axis=0, keepdims=True)
    # Uniform quantization to [0, levels-1]
    scaled = (x_rotated - lo) / (hi - lo + 1e-8) * (levels - 1)
    quantized = np.clip(np.round(scaled), 0, levels-1).astype(np.uint8)
    # Bit-pack: pack 'bits' values per byte
    # For 3-bit: 2 values per 6 bits + padding, or use np.packbits with
custom stride
    return _pack_bits(quantized, bits)  # returns (N, ceil(D*bits/8)) uint8
```

7.3 Fast dot product in compressed space

```
def compressed_dot(q_packed: np.ndarray, db_packed: np.ndarray,
                   bits: int = 3) -> np.ndarray:
    # q_packed: (ceil(D*bits/8),) uint8
    # db_packed: (N, ceil(D*bits/8)) uint8
    # Returns: (N,) float32 approximate dot products
    # Unpack and dequantize, then use numpy dot
    # For production: use a Cython/Numba kernel with SIMD popcount
    q = _unpack_bits(q_packed, bits).astype(np.float32)
    db = _unpack_bits(db_packed, bits).astype(np.float32)
    return db @ q  # (N,)
```

8. Graph Layer Implementation

8.1 Entity extraction prompt

The entity extraction step uses a structured prompt that forces JSON output. The prompt is designed to be model-agnostic — it works with GPT-4o, Claude, Gemini, and open models like Llama-3. The output schema is strict to enable reliable parsing without regex.

```
ENTITY_PROMPT = """
You are an entity extractor. Given the following text, extract all named
entities
and the relationships between them. Return ONLY valid JSON matching this
schema:
{
  "entities": [{"name": str, "type": str, "description": str}],
  "relationships": [{"source": str, "target": str, "relation": str, "weight":
float}]
```

```
}

Entity types: PERSON, ORG, CONCEPT, METRIC, DATE, PRODUCT, LOCATION

Relationship weight: 0.0 (weak) to 1.0 (strong/direct).

Text: {chunk_text}

"""
```

8.2 Community detection

After all chunks are processed, Leiden community detection runs on the NetworkX graph. Leiden is strictly better than Louvain for this use case — it guarantees that all communities are well-connected internally (no disconnected components), and it produces more stable results across runs. The leidenalg Python package wraps the C++ implementation.

```
import leidenalg

import igraph as ig


def detect_communities(G: nx.Graph, resolution: float = 1.0) -> Dict[str,
int]:

    # Convert NetworkX -> iGraph (leidenalg requires iGraph)

    ig_graph = ig.Graph.from_networkx(G)

    partition = leidenalg.find_partition(

        ig_graph,

        leidenalg.ModularityVertexPartition,

        resolution_parameter=resolution,

        seed=42

    )

    return {G.nodes[n]['name']: partition.membership[i]

            for i, n in enumerate(G.nodes())}
```

9. Comparison: TurboRAG vs PageIndex

PageIndex (VectifyAI, 11.6k GitHub stars) is the closest published system to TurboRAG in spirit — both aim to make RAG more accurate and more explainable. Understanding the differences precisely is important for positioning TurboRAG correctly and for knowing where the two systems could complement each other.

9.1 What PageIndex actually does

PageIndex is explicitly 'vectorless' — it abandons vector similarity search entirely. It uses an LLM to build a hierarchical tree index of a document (analogous to an intelligent table of contents), where each node summarises a section. Retrieval is done by LLM reasoning over this tree — the LLM reads the tree structure and decides which branches to follow. PageIndex achieves 98.7% accuracy on FinanceBench (financial document QA) by leveraging the LLM's reasoning rather than embedding similarity.

This is a fundamentally different bet: instead of making vector search better, PageIndex eliminates it. The tradeoff is that every query requires multiple LLM calls (to reason over the tree), and the index must be reconstructed from scratch per document (there is no persistent queryable index across a large corpus of heterogeneous documents).

9.2 Side-by-side comparison

Dimension	TurboRAG	PageIndex
Retrieval mechanism	Dense vector search (compressed)	LLM reasoning over tree index
Multi-hop reasoning	Via entity-relationship graph + community summaries	Via tree traversal guided by LLM
Corpus scale	Millions of documents, persistent index	Best for single/few long documents
Query latency	Milliseconds (compressed MIPS + graph BFS)	Seconds to tens of seconds (multiple LLM calls)

Index build cost	Embedding + compression + one-time graph extraction	Heavy: full LLM pass per document
Explainability	Graph path shown (entity chain to result)	Full reasoning trace from tree traversal
No-LLM mode	Yes — embedding + retrieval work without LLM at query time	No — LLM required for every query
Memory footprint	6× smaller than float32 baseline	No vectors — pure tree JSON
Update / incremental add	Yes — add shards without full re-index	Requires tree rebuild
Best for	Large-scale, low-latency, scalable production RAG	Deep analysis of single complex documents
Open source	Yes (TurboRAG — proposed)	Yes (MIT license)
Quantitative accuracy	Near-lossless vs float32 (JL guarantee)	98.7% on FinanceBench

9.3 The key architectural difference

PageIndex's tree is a document-structure index — it encodes how a document is organised. TurboRAG's graph is a semantic knowledge index — it encodes what concepts exist and how they relate, across all documents in the corpus. PageIndex cannot answer 'what did the CEO say about this topic across all earnings calls from 2022 to 2024' because it only indexes one document at a time. TurboRAG can, because the entity-relationship graph spans the entire corpus.

Complementary	PageIndex and TurboRAG are not competitors — they solve different problems. A future integration could use PageIndex for deep single-document analysis and TurboRAG for cross-document retrieval, routing queries to the appropriate subsystem based on scope.

10. Distribution Strategy

10.1 Primary: Python library on PyPI

TurboRAG ships as a pip-installable Python package. This is non-negotiable — the entire RAG ecosystem (LangChain, LlamaIndex, Haystack) is Python, and developers integrate libraries at the import level. The package has two install tiers:

```
pip install turborag              # core: compression + retrieval, no LLM
deps

pip install turborag[graph]       # adds: networkx, leidenalg, igraph

pip install turborag[serve]       # adds: fastapi, uvicorn

pip install turborag[all]         # everything
```

The core install has zero heavy dependencies — only numpy, scipy, and a sentence-transformers optional. This matters enormously for adoption: developers can try TurboIndex as a drop-in FAISS replacement without pulling in graph dependencies they may not want.

10.2 LangChain and LlamaIndex adapters

Both LangChain and LlamaIndex have formal VectorStore interfaces. TurboRAG ships adapters that implement these interfaces exactly, so any existing RAG pipeline can swap in TurboRAG with a one-line change:

```
# Before

from langchain.vectorstores import FAISS

db = FAISS.from_documents(docs, embeddings)



# After — one line change

from turborag.adapters.langchain import TurboVectorStore

db = TurboVectorStore.from_documents(docs, embeddings)
```

The adapter internally handles the compression step transparently. The developer gets 6× memory reduction and faster search with zero changes to the rest of their pipeline.

10.3 CLI tool

For non-programmatic use, TurboRAG ships a command-line interface:

```
turborag ingest --input ./docs/ --index ./my_index/ --graph

turborag query  --index ./my_index/ --query 'What is the capex guidance?'

turborag bench  --index ./my_index/ --baseline faiss --k 10

turborag serve  --index ./my_index/ --port 8080
```

The CLI makes TurboRAG accessible to ML engineers who want to evaluate it without writing any code, and to DevOps teams who want to run it as a sidecar service.

10.4 Docker image and self-hosted service

For teams that want to run TurboRAG as a persistent service (not embedded in their application), an official Docker image exposes a REST API:

```
docker pull ghcr.io/yourhandle/turborag:latest

docker run -v /data/index:/index -p 8080:8080 turborag serve
```

The REST API implements the same interface as the PageIndex API and the LlamaIndex vector store interface, maximising drop-in compatibility. Endpoints: POST /ingest (accepts document file or URL), POST /query (returns ranked results + graph path), GET /graph (returns community structure as JSON for visualisation).

10.5 MCP server

Given the rapid adoption of the Model Context Protocol for connecting AI agents to data sources, TurboRAG ships an MCP server that exposes turborag_query and turborag_ingest as tools. This enables any MCP-compatible agent (Claude, Cursor, any LangChain agent) to use TurboRAG as its retrieval backend with zero integration code. This is the distribution channel most likely to drive viral adoption among the agent developer community — the same channel that drove amcl-server to 8K+ downloads.

11. Benchmarking Plan

The README must ship with reproducible benchmark results. Without numbers, no serious engineer will swap their FAISS index for an unknown library. The benchmark suite covers three dimensions:

11.1 Recall accuracy

Dataset: BEIR benchmark (14 retrieval datasets covering news, biomedical, legal, financial). Metric: Recall@5, Recall@10, NDCG@10. Comparison: TurboRAG 3-bit vs TurboRAG 4-bit vs float32 FAISS (ground truth) vs IVF-PQ (standard compressed baseline). Expected result: TurboRAG 4-bit within 0.5% NDCG@10 of float32 baseline; IVF-PQ 5-15% worse depending on dataset.

11.2 Memory footprint

Dataset: 1M, 5M, 10M vectors at D=768, D=1536, D=3072. Metric: peak RSS during index load and during search. Comparison: flat float32, HNSW (faiss.IndexHNSWFlat), IVF-PQ, TurboRAG 3-bit. Expected: TurboRAG 5-7× lower than float32 baseline, 12-18× lower than HNSW.

11.3 Query throughput

Metric: queries per second at batch size 1 (latency-sensitive), batch size 32 (throughput-optimised). Hardware: both CPU-only (AMD EPYC, common in cheap cloud) and GPU (A100). Expected: TurboRAG faster than HNSW at CPU for N > 1M due to cache efficiency of compact representations; slower for N < 100K.

11.4 End-to-end RAG quality

Dataset: FinanceBench (same as PageIndex benchmark) and MultiHop-RAG. Metric: exact match accuracy on QA pairs. Comparison: flat FAISS + no graph, TurboRAG dense only, TurboRAG hybrid (dense + graph). This is the headline number. Expected: TurboRAG hybrid within 2% of PageIndex on FinanceBench, dramatically better than flat FAISS on multi-hop questions.

12. Agent Build Instructions

The following is a sequenced set of tasks for a coding agent to implement TurboRAG in order of dependency. Each task is self-contained and testable.

Phase 1 — Core compression (no LLM dependency)

13. Implement generate_rotation(dim, seed) -> np.ndarray using scipy.stats.ortho_group for dim ≤ 2048 and block-diagonal rotation for larger dims. Unit test: verify R @ R.T ≈ I within float32 tolerance.
14. Implement quantize_qjl(x, bits) -> np.ndarray and dequantize_qjl(x_packed, bits) -> np.ndarray. Unit test: verify round-trip error is within JL bound for random vectors.
15. Implement compressed_dot(q, db, bits) -> np.ndarray. Unit test: compare output to numpy.dot on dequantized vectors; verify error < 5% for random unit vectors.
16. Implement TurboIndex class with in-memory storage (no sharding yet). Test: add 10K vectors, search returns correct top-10 vs brute-force.
17. Add shard-based on-disk storage with numpy memmap. Test: index 1M vectors across shards, reload from disk, verify search results match in-memory version.

Phase 2 — Graph layer (requires LLM client)

18. Implement entity extraction using the structured prompt in Section 8.1. Use a mock LLM returning hardcoded JSON for unit tests. Real integration test against a small document with GPT-4o-mini.
19. Implement GraphBuilder.add_chunk() and GraphBuilder.build(). Test: 20-chunk document produces a connected graph with at least 5 entities.
20. Implement Leiden community detection wrapper. Test: synthetic graph with two clear clusters produces 2 communities.
21. Implement community summarisation with LLM caching. Test: re-running on same corpus hits cache 100% (no new LLM calls).

Phase 3 — Hybrid retrieval

22. Implement HybridRetriever.query() with mode='dense'. Test: matches TurboIndex.search() output exactly.
23. Implement graph routing: entity detection in query string → BFS on entity graph → collect related community IDs. Test: query containing a known entity returns results from its community.
24. Implement cross-encoder re-ranking. Test: re-ranked results have higher MRR@10 than dense-only on a held-out validation set.
25. Implement HybridRetriever.explain(). Test: every returned result has a non-empty explanation string.

Phase 4 — Ingest pipeline and CLI

26. Implement document chunking with tiktoken for token counting. Handle PDF (via pdfminer.six), markdown, and plain text.
27. Implement the turborag CLI using Click or Typer. Test: ingest a 10-page PDF, query it, get results in < 500ms on CPU.
28. Write the turborag.bench module. Test: outputs a markdown table comparing TurboRAG vs FAISS on a 100K-vector subset of MS MARCO.

Phase 5 — Adapters and distribution

29. Implement LangChain VectorStore adapter. Test: swap into an existing LangChain QA chain, verify answers match.
30. Implement LlamaIndex VectorStore adapter. Same test.
31. Implement FastAPI serve module. Test: POST /ingest with a PDF, then POST /query, verify JSON response schema.
32. Implement MCP server using the MCP Python SDK. Test: expose turborag_query as a tool, call it from a Claude agent.
33. Write pyproject.toml with optional dependency groups. Publish to TestPyPI. Verify pip install turborag installs with only numpy/scipy as heavy deps.

13. Dependency Manifest

Package	Purpose
numpy >= 1.24	Array operations, memmap storage
scipy >= 1.10	Orthogonal group sampling (rotation generation)
networkx >= 3.0	Graph construction and serialisation [graph extra]
leidenalg >= 0.10	Leiden community detection [graph extra]
python-igraph >= 0.10	Required by leidenalg [graph extra]
sentence-transformers >= 2.5	Default local embedding model [optional]

pdfminer.six >= 20221105	PDF text extraction [ingest extra]
fastapi >= 0.110	REST API server [serve extra]
uvicorn >= 0.27	ASGI server [serve extra]
click >= 8.0	CLI interface
mcp >= 1.0	MCP server [mcp extra]
tiktoken >= 0.6	Token counting for chunking

14. Open Research Questions

These are the directions where TurboRAG extends beyond engineering into publishable research. Each is a genuine open question with no published answer.

- Domain-adaptive rotation: TurboQuant uses a random rotation (Haar-distributed). For domain-specific corpora (financial, biomedical), a calibrated rotation tuned to the corpus distribution should achieve equivalent recall at lower bit-width. The experiment: run 1K calibration vectors through PCA, use the PCA basis as R, compare recall vs random R at 2-bit quantization.
- Per-layer bit allocation: different layers of a transformer's KV cache have different sensitivity to quantization. The same principle applies to the graph layer in TurboRAG — entity embeddings (high precision needed for graph routing) vs community summary embeddings (lower precision acceptable). Implement mixed-bit allocation with a budget constraint and measure the Pareto frontier.
- Incremental graph maintenance: the current design rebuilds the graph when new documents are added. Real production systems ingest documents continuously. Research question: can the Leiden partition be updated incrementally without full re-clustering? leidenalg has a partition update API that enables this but it has not been evaluated for RAG quality.
- Retrieval-aware quantization: current quantization is retrieval-agnostic — it minimises reconstruction error uniformly. A future direction is to minimise retrieval error instead — specifically, to minimise the probability that a correct result is displaced from the top-k by quantization noise. This requires a differentiable proxy for recall@k, which is non-trivial but tractable.

15. Project Naming and Positioning

The library name TurboRAG is available on PyPI as of March 2026. It is short, memorable, directly descriptive, and connects to TurboQuant's brand. The GitHub repo name turborag is also available.

Positioning statement for the README: 'TurboRAG is the first retrieval library to combine near-lossless vector compression (TurboQuant, Google, ICLR 2026) with knowledge-graph-augmented retrieval. It fits in 6× less memory than a standard vector store, runs faster than HNSW on large corpora, and answers multi-hop questions that pure embedding search cannot.'

This positioning is differentiated from every current alternative. FAISS is fast but uncompressed. Chroma and Pinecone are uncompressed managed stores. LlamaIndex and LangChain are frameworks, not storage engines. PageIndex is vectorless (different problem). GraphRAG is graph-only without compression. TurboRAG is the only system that is simultaneously compressed, fast, accurate, and graph-augmented.

End of Specification

TurboRAG — Ratnam — March 2026